Ders 21 Çalışma Özeti

Ders 21 Çalışma Özeti

Genel Konular

- Bu kayıt aynı günün (24.05.2021) iki bölümünü içerir:
  - Üçüncü laboratuvardan sonra yapılan bir saatlik ders: özyinelemeli (recursive) algoritmaların analizi, recurrence bağıntılarının recursion tree ile çözülmesi, sınav ve projeyle ilgili duyurular.
  - Kaydın ikinci parçası, öğle saatindeki Laboratuvar 3'ün açılışı: haritada en kısa yol sorusu.
- Laboratuvar 3 sorusu üzerine konuşma
  - Bir öğrenci, yolun her adımda sağdaki ve aşağıdaki hücrelerden küçüğünü seçerek mi yoksa bütün yolların toplamına bakarak mı bulunacağını sordu. Hoca, her adımda en küçüğü seçmenin her zaman en kısa toplam yolu vermediğini, yolun tamamına bakmak gerektiğini söyler.
  - Hoca bu problemi kombinatorikteki yol sayma (path counting) problemine benzetir. Örnek olarak 5 bitle 2^5 farklı bilgi saklanabildiğini hatırlatır. Bu tür problemlerde olası durum sayısı 2^n, $n!$, n^n gibi polinom olmayan hızlarda artar. Bu konu dönemin son dersinde ve sonraki derslerde (ayrık matematik, veri yapıları, algoritma analizi) işlenecek.
  - Hoca problem ile algoritmanın farklı kavramlar olduğunu hatırlatır: algoritma, bir problemin çözüm yöntemidir.
- Özyinelemeli algoritmaların analizi: genel yöntem
  - İteratif algoritmalarda işlemler toplam veya çarpım sembolüyle sayılıyordu. Özyinelemeli algoritmalarda problem kendini daha küçük parçalarla tekrar eder, bu yüzden farklı bir yol izlenir.
  - Kabaca: karmaşıklık ≈ (bir çağrıdaki temel işlemin maliyeti) × (özyinelemeli çağrı sayısı).
  - Temel işlem, her çağrıda kesin yapılan işlemdir. Faktöriyelde `n == 1` karşılaştırması her çağrıda yapılır, atama ise yalnızca en sonda yapılır. Karşılaştırma, toplama, çıkarma, atama gibi işlemler makine dilinde tek komuttur (`CMP`, `ADD`, `SUB`) ve 1 birim sayılır. Çağrının içinde bir döngü veya başka bir fonksiyon varsa temel işlemin maliyeti 1 değil, o döngünün veya fonksiyonun maliyeti olur.
  - Sabit sayıda işlem (1, 2, 5, 10) her zaman sabit (1) kabul edilir.
  - Ders notundaki sözde kodda `T = ...` satırı fonksiyonun dönüş değerini gösterir (MATLAB ve Python'daki gibi).
- Backward substitution ile çağrı sayısı
  - Faktöriyel: n, $n-1$'i çağırır, o da $n-2$'yi... k adım sonra $n-k$'ye gelinir. Durma koşulu $n-k=1$ olduğundan $k=n-1$. Toplam yaklaşık n çağrı, karmaşıklık n.
  - Üs alma `x * US(x, n-1)`: her çağrıda bir karşılaştırma ve bir çarpma var (sabit). `x` değişmez, `n` birer azalır. Karmaşıklık n.
  - `f(a, b) = f(a+1, b-1)`: durmayı `b` kontrol eder, `a` etkilemez. $b-k=0$ olduğunda $k=b$. Karmaşıklık b, yani girdi boyutuna bağlı doğrusal (n).
  - Dizinin elemanlarını özyinelemeli toplama da n'dir (öğrencilere bırakıldı).
- Özyinelemeli sıralı arama: en iyi, ortalama, en kötü durum
  - Dizinin sonundan başa doğru aranır. İki olasılık vardır: eleman bulunur veya bulunamaz.
  - En iyi durum (best case): ilk bakılan elemanda bulunur. Bir karşılaştırma ve bir özyineleme kontrolü, sabit zaman.

  - Ortalama durum (average case): arada bir yerde bulunur. Bu ders kapsamında ayrıntısına girilmedi.
  - En kötü durum (worst case): bulunamaz, dizinin başına kadar gidilir. Her çağrıda 2 işlem, b çağrı, yani $2b$, karmaşıklık n.
  - Karmaşıklıklar Big-O ($O(n)$) gösterimiyle yazılır. Big-Theta ve Big-Omega gelecek dönem ayrık matematikte görülecek.
- Problemi bölerek ilerleyen algoritmalar
  - Her çağrı bir eksiğini değil, yarısını, üçte birini veya başka bir oranını da çağırabilir. Örnek: özyinelemeli ikili arama.
  - Hızlı üs alma (`temp`'li sürüm): p, $p/2$, $p/4$, ... $p/2^k$. Tam sayı bölmesiyle sonunda 0'a inilir, ama k'yı bulmak için $\frac{p}{2^k} = 1$ alınır (0'a eşitlemek çözüm vermez). $k = \log_2 p$. Hoca gerçek çağrı sayısı için $\log_2 p + 1$ dedi; p 2'nin kuvveti iken 0'a inene kadarki çağrı sayısı $\log_2 p + 2$'dir (1024 için 12). Sabit fark atılır. Karmaşıklık $\log n$. Döngüyle n çarpma yapılıyordu, bu çok daha iyi bir algoritmadır.
  - Aynı fonksiyonun `temp` olmadan iki kez çağrıldığı sürüm (`US(x, p/2) * US(x, p/2)`): ağaç her düzeyde ikiye dallanır. Düzeylerde $2^1, 2^2, \ldots, 2^h$ düğüm vardır ve $h = \log_2 n$. Zaten hesaplanmış alt problemler yeniden hesaplanır. Toplam iş n'ye çıkar.
- Merge sort'un analizi
  - Bir animasyonla adım adım gösterildi: dizi baş ve son indislere göre ortadan ikiye bölünür, tek elemanlı parçalara kadar inilir. Sonra iki sıralı parça yardımcı bir diziye yedeklenip birleştirilir (merge) ve orijinal diziye yazılır.
  - Her çağrı $n/2$ boyutunda iki özyinelemeli çağrı yapar. Birleştirme n işlem gerektirir.
  - Her düzeyde toplam iş n (2 tane $n/2$, 4 tane $n/4$, ...). Derinlik $\frac{n}{2^h} = 1$ eşitliğinden $h = \log_2 n$. Toplam $n \log n$. Bu, n^2 sıralama algoritmalarından çok daha iyidir.
- Recurrence bağıntılarından karmaşıklık (Fatih Hoca'nın ders notu, Ders 9)
  - $T(n) = 2T(n/4) + n$: kod kendini iki kez, her seferinde dörtte bir boyutla çağırır. n, kodun bir çağrıda yaptığı iştir. Düzey maliyetleri: n, $2 \cdot \frac{n}{4} = \frac{n}{2}$, $4 \cdot \frac{n}{16} = \frac{n}{4}$, ... Derinlik $h = \log_4 n$. $T(n) = n \sum_{i=0}^{\log_4 n} \left(\frac{1}{2}\right)^i = 2n - \sqrt{n}$. Baskın terim $2n$, karmaşıklık n. $2n$, $\sqrt{n}$'den hızlı büyür.
  - $T(n) = 8T(n/2) + n$: düzey maliyetleri n, $8 \cdot \frac{n}{2} = 4n$, $64 \cdot \frac{n}{4} = 16n$, ... Derinlik $\log_2 n$. $T(n) = n \sum_{i=0}^{\log_2 n} 4^i = \frac{n(4 \cdot 4^{\log_2 n} - 1)}{3}$. $4^{\log_2 n} = 2^{2 \log_2 n} = n^2$ olduğundan sonuç n^3 mertebesindedir.
  - $T(n) = 8T(n/2) + n^2$: düzey maliyetleri n^2, $8 \cdot \frac{n^2}{4} = 2n^2$, $4n^2$, ... $T(n) = n^2 \sum_{i=0}^{\log_2 n} 2^i = n^2(2n - 1)$. Sonuç yine n^3.
  - Hoca, ders notundaki diğer soruların ertesi günkü ek derse kadar çalışılmasını ister.
- Proje duyuruları
  - Reddedilen önerilerin çoğunda sorun şu: bir problem önerilmiş, algoritma değil. Örneğin "bir sayının asal çarpanlarını bulan program" bir algoritma adı değildir. "Asal çarpanları falanca algoritmayla bulmak" denmeli ve bu algoritma derste görülmemiş olmalı. Basit bir `for` döngüsüyle yapılan işler de kabul edilmez. Insertion sort, bubble sort gibi derste görülen algoritmalar reddedilir.
  - Sıfırdan algoritma tasarlanmayacak, var olan bir algoritma dersin ölçütleriyle analiz edilecek.
  - Öneriler, algoritmanın zorluğuna ve daha önce kabul edilen önerilere göre seçilir. Geçen yılın kabul edilmiş projeler listesi de dikkate alınır. Öneri sırası garanti değildir. İki grubun önerileri tek havuzdadır.
  - Video: tek bir video, 5-7 dakika. Başta yüz görünmeli, sonra ekran kaydı ve ses olmalı. Sessiz veya görüntüsüz video değerlendirilmez. YouTube yerine Google Drive linki de olur. Videolar final belgesi olarak saklanacak (MÜDEK kayıtları için birkaç örnek indirilecek).
  - Raporda zaman ve bellek açısından farklı performans ölçütleri olmalı. Kod içinde sonuçlar otomatik görselleştirilmeli. C'de bunun basit yolu, ölçeklenmiş değerleri yıldız (*) karakterleriyle histogram gibi çizmektir.
  - Zaman ve bellekten en az biri incelenmeli. Hangisinin anlamlı olduğu algoritmaya göre değişir. Örneğin merge sort ve quick sort ikisi de $n \log n$'dir. Merge sort ek bellek kullandığı için karşılaştırmada bellek açısı önem kazanır.
  - Raporda kullanılan kaynaklar yazılmalıdır.

- Laboratuvar 3 (öğle saatinde, asistanlar tarafından)
  - Bir harita (kare matris) verilir. Başlangıç ve bitiş noktası kullanıcıdan alınır ve ikisi de haritada 0 olan hücreler olmalıdır. İki nokta arasındaki en düşük maliyetli yol bulunur. Her hücredeki değer o hücreye girmenin maliyetidir (negatif olabilir).
  - Hareketler sağ, sol, yukarı, aşağı. Çapraz yok. Yol hedefe doğru ilerler. Örnekte (1,1)'den (4,4)'e giderken yalnızca sağ ve aşağı hareket edilir. Hedef yukarıda veya solda olabilir, o zaman hareketler ona göre değişir.
  - Harita koda doğrudan yazılabilir, kullanıcıdan alınmaz. Boyut `N` kullanıcıdan alınmalıdır. Kod sabit 5 × 5'e göre değil, 8 × 8 gibi başka boyutlarda da çalışacak şekilde yazılmalıdır.
  - Başlangıç veya bitiş 0 değilse hata mesajı verilip çıkılabilir veya yeniden sorulabilir. Kullanıcıya her adımda açıklayıcı mesajlar gösterilmeli, boş ekranda girdi beklemek puan kaybettirir.
  - Çıktı: toplam maliyet ve hamleler (1 sağ, 2 sol, 3 yukarı, 4 aşağı). Örnekte (1,1)'den (4,4)'e en kısa yolun maliyeti -55, rota 1 4 4 1 4 1.
  - Fonksiyon ve struct kullanılabilir. Teslim Google Form ile, bitiş 15.35. Asistana göre sorunun bir kritik noktası var ve o bulunursa 10 dakikada çözülür.

Hocanın Özellikle Vurguladığı Kısımlar

- Özyinelemeli algoritmada önce temel işlemi ve maliyetini bulun, sonra çağrı sayısını bulun.
- Çağrı sayısı için problemi geriye doğru açın ($n, n-1, \dots, n-k$ veya $n, n/2, \dots, n/2^k$) ve durma koşuluna eşitleyerek k'yı bulun.
- Bir çağrıda kaç özyinelemeli çağrı yapıldığına dikkat edin. İki çağrı ağacı dallandırır.
- Proje önerisinde problem değil, algoritmanın adını yazın.
- Videoda yüzünüz ve sesiniz olmalı.

Kısa Tekrar Notları

- Faktöriyel, basit üs, `f(a+1, b-1)`: n.
- Özyinelemeli sıralı arama: en iyi 1, en kötü n.
- Hızlı üs (`temp` ile) ve ikili arama: $\log n$.
- Merge sort: $T(n) = 2T(n/2) + n$, $n \log n$.
- $2T(n/4) + n = 2n - \sqrt{n}$, yani n. $8T(n/2) + n$ ve $8T(n/2) + n^2$: n^3.
- $4^{\log_2 n} = n^2$, $2^{\log_2 n} = n$.

Detaylı Açıklamalar

Laboratuvar 3 için dinamik programlama çözümü

Aşağıdaki çözüm derste yazılmadı. Ders sonrasında hazırlandı ve `gcc -Wall -std=c89 -pedantic` ile denendi. Hedefe doğru ilerleyen her hücreye en ucuz yol, ya önceki satırdaki ya da önceki sütundaki komşudan gelir. Bu yüzden her hücre için iki komşunun en küçüğü alınır (dinamik programlama):

```
int enKisa(int sx, int sy, int hx, int hy){
    int dp[MAX][MAX], dx, dy, i, j, ni, nj, a, b, ri, rj;
    dx = hx > sx ? 1 : -1;            /* satir yonu */
    dy = hy > sy ? 1 : -1;            /* sutun yonu */
    ri = (hx - sx) * dx;
    rj = (hy - sy) * dy;
    for (i = 0; i <= ri; i++)
        for (j = 0; j <= rj; j++) {
            ni = sx + i * dx;
            nj = sy + j * dy;
            if (i == 0 && j == 0) { dp[i][j] = 0; continue; }
            a = i > 0 ? dp[i-1][j] : 1000000;
            b = j > 0 ? dp[i][j-1] : 1000000;
            dp[i][j] = (a < b ? a : b) + harita[ni][nj];
```

```
        }
    return dp[ri][rj];
}
```

Laboratuvardaki 5 × 5 harita için (1,1)'den (4,4)'e sonuç -55 çıktı. Bu, soru metnindeki örnek çıktıyla aynıdır. Rotayı yazdırmak için her hücrede hangi komşudan gelindiği ayrıca saklanabilir. `continue` bu derste yasaktır, sınavda `if-else` ile yazılmalıdır.

Ders sonrası notlar

- **Yol sayısı:** Hoca 7 hamlelik yolların sayısını "ilk 4, sonraki 3" diyerek $4! \cdot 3!$ olarak hesapladı (büyük olasılıkla 4 sağ ve 3 aşağı hamle kastedildi). Bu, sağ hareketleri ve aşağı hareketleri birbirinden ayırt edilebilir saymak demektir. Hareketler kendi aralarında aynı olduğu için doğru sayı $\frac{7!}{4!\,3!} = \binom{7}{3} = 35$'tir. Öğrencinin söylediği 70 de 5 × 5 haritada köşeden köşeye ($4+4$ hamle) yol sayısıdır: $\binom{8}{4} = 70$.
- **Bu problem polinom zamanda çözülür:** Hoca bu problemi polinom zamanda çözülemeyen problemlere benzetti. Bütün yolları denemek gerçekten üstel sayıda yol demektir. Ama hareketler hedefe doğru sınırlıyken yukarıdaki dinamik programlama çözümü her hücreye bir kez bakar ve satır × sütun (n^2) adımda biter. Asistanın söylediği "kritik nokta" büyük olasılıkla budur. Dört yöne serbest hareket edilebilen genel durumda da negatif döngü yoksa en kısa yol polinom zamanda bulunur (Bellman-Ford algoritması).
- **Her adımda en küçüğü seçmek (greedy):** Hocanın dediği gibi bu yöntem en iyi sonucu garanti etmez. Laboratuvar haritasında (1,1)'den (4,4)'e her adımda sağ ve aşağıdan küçüğünü seçmek de -55 veriyor, ama bu bu haritaya özgü bir tesadüftür.
- **İki çağrılı üs almanın toplamı:** Hoca toplam işi $\log n \cdot 2^{\log n}$ gibi ifade etti. Bu çarpım $n \log n$ eder. Doğrusu düzeylerin toplamıdır: $\sum_{i=0}^{\log_2 n} 2^i = 2n - 1$. Hocanın vardığı n sonucu doğrudur.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.